

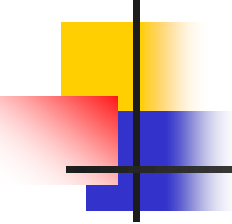
CS501

Introductory Computation for Scientists



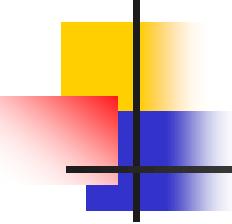
Dr. Ash Pahwa

Lesson 7.1: Pandas Introduction



Outline

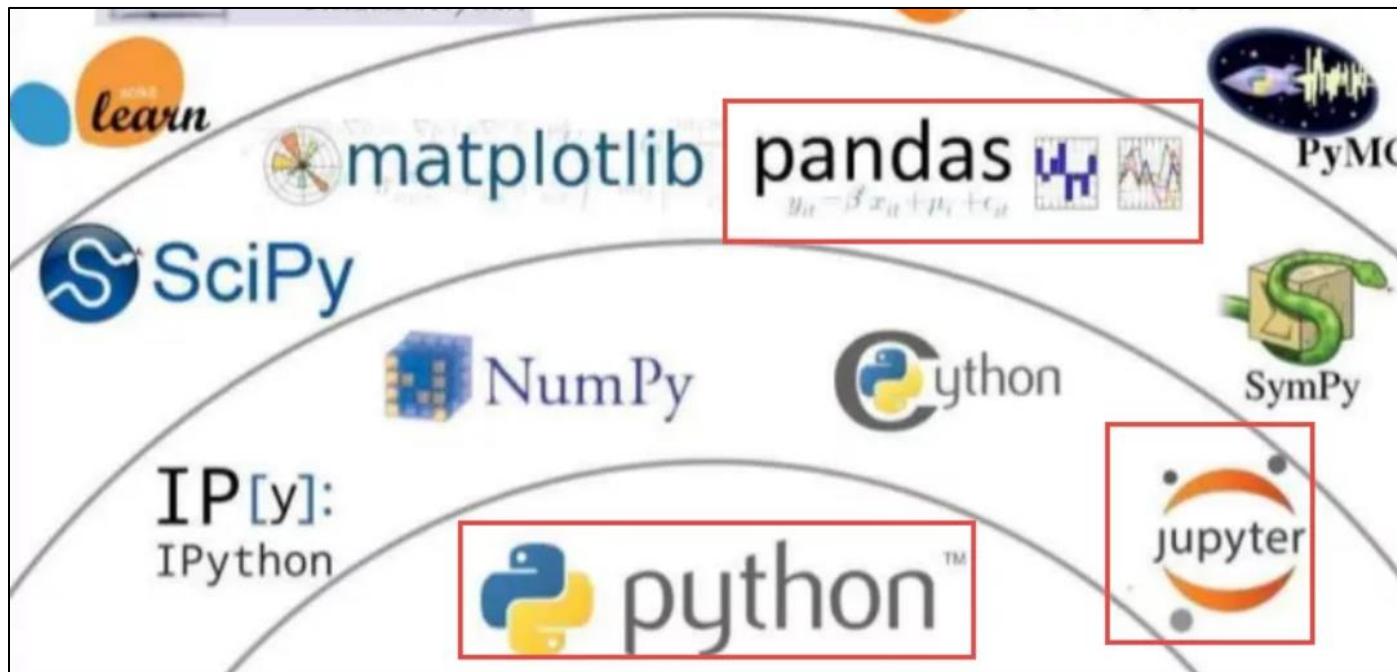
- What is Pandas?
- Pandas Functions
- Why Pandas?
- Dataset + Load Data in Pandas' Data Frame
- Read Data from a Data Frame
- Sorting Data
- Make Changes to Data
- Saving Data Frame to a File
- Filter Data Select Boolean Conditions
- Groupby Clause (Pivot Table)



What is Python's Pandas

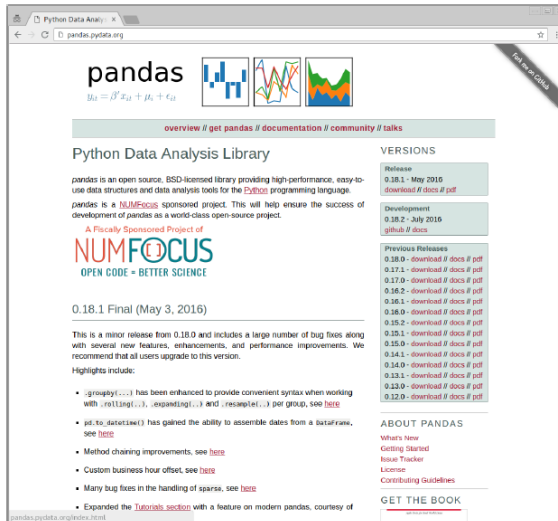
- **Pandas** is a software library written for the **Python** programming language for data manipulation and analysis.
- It offers data structures and operations for manipulating numerical tables and time series.

Pandas



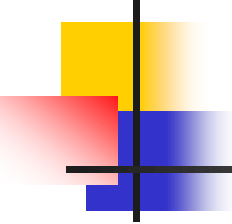
Author: Wes McKinney

Pandas



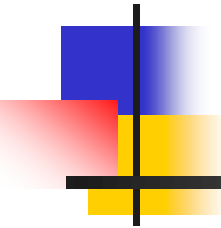
- Created in 2008 by Wes McKinney
- Open source New BSD license
- 100 different contributors

Name: Name of the package "Pandas" comes from "Panel Data" (time series data)

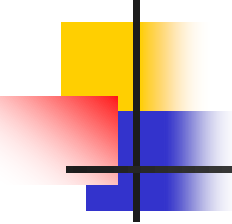


Python Data Structures

	Package • Data Structure	Create	Index Start=0	Slice	Edit
1	Core Python • String	s='abcdefghi'	s[2]	s[start:end:inc] s[2:8:2]	Immutable
2	Core Python • List	1D: l = [1,2,3] 2D: l = [1,2,list1]	1D: l[2] 2D: l[2][3]	l[start:end:inc] l[2:8:2]	Mutable
3	Core Python • Tuples	1D: t = (1,2,3) 2D: t = (1,2,list)	1D: t[2] 2D: t[2][3]	t[start:end:inc] t[2:8:2]	Immutable
4	Core Python • Dictionary	1D: d = {1:1, 2:2, 3:3 } 2D: d = {1:1, 2:2, 3:list }	Indexing Using keys		Mutable
5	Numpy • Arrays	np.array(List)	1D: a[2] 2D: a[2][3]	a[start:end:inc] a[2:8:2]	Mutable
6	Pandas • Series • DataFrame	1D: pd.Series(<index>,<data>) 2D: pd.DataFrame(<s1>,<s2>, ..)	s.loc[<index>] s.iloc[2] df.loc['index']['c1','c2']		Mutable
7	Datascience • Table	2D: t1 = Table().with_column('col name', <data>, ...)	2D: t1.select('col name')	2D: t1.where('con dition')	Mutable

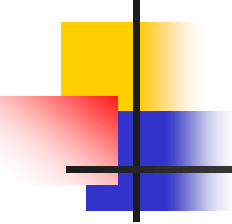


Pandas Functions



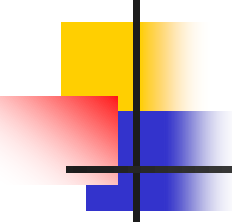
Create & Modify Data Structure

- Read a tabular data structure from a csv file
 - Data Frame is created using the structure of the data
- Change Data Frame Data Structure
 - Add a Column
 - Delete a column
 - Modify a column



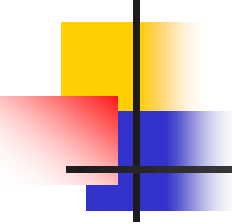
Data Retrieval

- Retrieve data based on columns
- Retrieve data based on rows
- Retrieve data based on some Boolean condition
- Sort the retrieve data
- Group by data with categorical variable
 - Pivot table (Excel)
- Iterate through each row/column



Printing Data

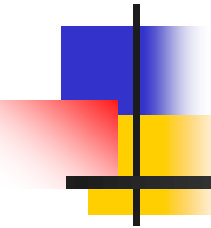
- Print data from a single column
- Print data from multiple columns
- Print rows using
 - `.loc`
 - `.iloc`



Data Editing

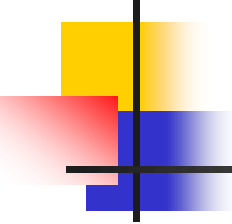
Insertion + Deletion + Modification

- Add a row
- Delete a row
- Change a data element of a row & column



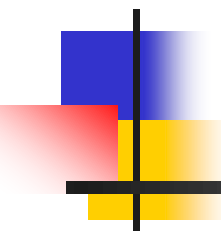
Why Pandas?

Comparison with Excel



Why Pandas?

- Pandas is like Microsoft's Excel
- Whatever you can do in Excel
 - You can do it Python
- Pandas provides better performance with big data
- Pandas is more flexible than Excel



Dataset

Dataset:

pokeman_data.csv (Comma Separated)

Tab Separated

Excel format

■ Available from Kaggle

	A	B	C	D	E	F	G	H	I	J	K	L
1	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
2	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	FALSE
3	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	FALSE
4	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	FALSE
5	3	VenusaurM	Grass	Poison	80	100	123	122	120	80	1	FALSE
6	4	Charmand	Fire		39	52	43	60	50	65	1	FALSE
7	5	Charmelec	Fire		58	64	58	80	65	80	1	FALSE
8	6	Charizard	Fire	Flying	78	84	78	109	85	100	1	FALSE
9	6	CharizardM	Fire	Dragon	78	130	111	130	85	100	1	FALSE
10	6	CharizardM	Fire	Flying	78	104	78	159	115	100	1	FALSE
11	7	Squirtle	Water		44	48	65	50	64	43	1	FALSE
12	8	Wartortle	Water		59	63	80	65	80	58	1	FALSE
13	9	Blastoise	Water		79	83	100	85	105	78	1	FALSE
14	9	BlastoiseM	Water		79	103	120	135	115	78	1	FALSE
15	10	Caterpie	Bug		45	30	35	30	30	45	1	FALSE

Name	Status	Date modified	Type	Size
 pokemon_data.csv		9/16/2021 2:48 PM	Microsoft Excel Co...	40 KB
 pokemon_data.txt		9/16/2021 2:48 PM	Text Document	40 KB
 pokemon_data.xlsx		9/16/2021 2:53 PM	Microsoft Excel W...	59 KB

Load Data into Pandas

- Read a .csv file
- Read Excel spreadsheet file
- Read a tab separated file

```
[1] import pandas as pd
```

```
[3] from google.colab import files
files.upload()
```

Choose Files 2 files

- **pokemon_data.txt**(text/plain) - 40832 bytes, last modified: 9/16/2021 - 100% done
 - **pokemon_data.xlsx**(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)
- Saving pokemon_data.txt to pokemon_data.txt
Saving pokemon_data.xlsx to pokemon_data.xlsx
{'pokemon_data.txt': b'#\tName\tType 1\tType 2\tHP\tAttack\tDefense\tSp. A
'pokemon_data.xlsx': b'PK\x03\x04\x14\x00\x06\x00\x08\x00\x00!\x00b\x00'}

```
df = pd.read_csv('pokemon_data.csv')
print(df.head(5))
print(df.tail(5))
```

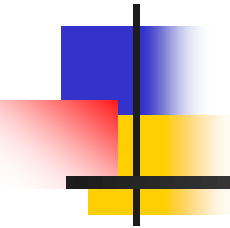
```
# df_xlsx = pd.read_excel('pokemon_data.xlsx')
# df = pd.read_csv('pokemon_data.txt', delimiter='\t')
```

#		Name	Type 1	...	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	...	45	1	False
1	2	Ivysaur	Grass	...	60	1	False
2	3	Venusaur	Grass	...	80	1	False
3	3	VenusaurMega	Grass	...	80	1	False
4	4	Charmander	Fire	...	65	1	False

[5 rows x 12 columns]

#		Name	Type 1	...	Speed	Generation	Legendary
795	719	Diancie	Rock	...	50	6	True
796	719	DiancieMega	Rock	...	110	6	True
797	720	HoopaHoopa Confined	Psychic	...	70	6	True
798	720	HoopaHoopa Unbound	Psychic	...	80	6	True
799	721	Volcanion	Fire	...	70	6	True

[5 rows x 12 columns]



Read Data From a Data Frame

Read Column Names

Read Specific Column

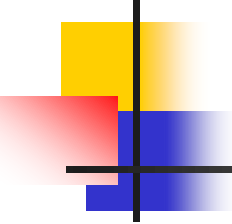
✓ [15] ##### Read Headers
0s df.columns

```
Index(['#', 'Name', 'Type 1', 'Type 2', 'HP', 'Attack', 'Defense', 'Sp. Atk',  
      'Sp. Def', 'Speed', 'Generation', 'Legendary'],  
      dtype='object')
```

✓ ## Read each Column
0s print(df[['Name', 'Type 1', 'HP']])

	Name	Type 1	HP
0	Bulbasaur	Grass	45
1	Ivysaur	Grass	60
2	Venusaur	Grass	80
3	VenusaurMega Venusaur	Grass	80
4	Charmander	Fire	39
..	...	...	..
795	Diancie	Rock	50
796	DiancieMega Diancie	Rock	50
797	HoopaaHoopaa Confined	Psychic	80
798	HoopaaHoopaa Unbound	Psychic	80
799	Volcanion	Fire	80

[800 rows x 3 columns]



Data Indexing: loc & iloc

- loc- uses labels but works with Boolean array as well. In other words, this works better with column names.
- iloc- uses integer index position or Boolean array.

Read Data: iloc Method

Read Each Row

loc: Indexing

iloc: Integer Location

```
## Read Each Row
print(df.iloc[0:4])
for index, row in df.iterrows():
    print(index, row['Name'])
```

#		Name	Type	1	...	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	...	45	1	False	
1	2	Ivysaur	Grass	...	60	1	False	
2	3	Venusaur	Grass	...	80	1	False	
3	3	VenusaurMega Venusaur	Grass	...	80	1	False	

[4 rows x 12 columns]

0 Bulbasaur

1 Ivysaur

2 Venusaur

3 VenusaurMega Venusaur

4 Charmander

5 Charmeleon

6 Charizard

7 CharizardMega Charizard X

8 CharizardMega Charizard Y

Read From a Specific Location loc Method

```
df.loc[df['Type 1'] == "Grass"]
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
48	43	Oddish	Grass	Poison	45	50	55	75	65	30	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...
718	650	Chespin	Grass	NaN	56	61	65	48	45	38	6	False
719	651	Quilladin	Grass	NaN	61	78	95	56	58	57	6	False
720	652	Chesnaught	Grass	Fighting	88	107	122	74	75	64	6	False
740	672	Skiddo	Grass	NaN	66	65	48	62	57	52	6	False
741	673	Gogoat	Grass	NaN	123	100	62	97	81	68	6	False

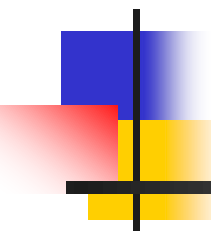
70 rows × 12 columns

+ Code

+ Text

```
## Read a specific location (R,C)  
print(df.iloc[2,1])
```

Venusaur



Sorting Data

Describe:

Compute Characteristics of Numerical Data

	A	B	C	D	E	F	G	H	I	J	K	L
1	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
2	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	FALSE
3	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	FALSE
4	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	FALSE
5	3	VenusaurM	Grass	Poison	80	100	123	122	120	80	1	FALSE



```
df.describe()
```

	#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation
count	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000
mean	362.813750	69.258750	79.001250	73.842500	72.820000	71.902500	68.277500	3.32375
std	208.343798	25.534669	32.457366	31.183501	32.722294	27.828916	29.060474	1.66129
min	1.000000	1.000000	5.000000	5.000000	10.000000	20.000000	5.000000	1.00000
25%	184.750000	50.000000	55.000000	50.000000	49.750000	50.000000	45.000000	2.00000
50%	364.500000	65.000000	75.000000	70.000000	65.000000	70.000000	65.000000	3.00000
75%	539.250000	80.000000	100.000000	90.000000	95.000000	90.000000	90.000000	5.00000
max	721.000000	255.000000	190.000000	230.000000	194.000000	230.000000	180.000000	6.00000

Sorting on a Single Column

Default: Ascending (A ... Z)

df.sort_values(['Name'])

#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
510 460	Abomasnow	Grass	Ice	90	92	75	92	85	60	4	False
511 460	AbomasnowMega Abomasnow	Grass	Ice	90	132	105	132	105	30	4	False
68 63	Abra	Psychic	NaN	25	20	15	105	55	90	1	False
392 359	Absol	Dark	NaN	65	130	60	75	60	75	3	False
393 359	AbsolMega Absol	Dark	NaN	65	150	60	115	60	115	3	False
...	...	...	...	...	...	...	...	...	...	...	...
632 571	Zoroark	Dark	NaN	60	105	60	120	60	105	5	False
631 570	Zorua	Dark	NaN	40	65	40	80	40	65	5	False
46 41	Zubat	Poison	Flying	40	45	35	30	40	55	1	False
695 634	Zweilous	Dark	Dragon	72	85	70	65	70	58	5	False
794 718	Zygarde50% Forme	Dragon	Ground	108	100	121	81	95	95	6	True

800 rows x 12 columns

Sorting: Descending Order (Z ... A)

```
df.sort_values(['Name'],ascending=False)
```

#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
794 718	Zygarde50% Forme	Dragon	Ground	108	100	121	81	95	95	6	True
695 634	Zweilous	Dark	Dragon	72	85	70	65	70	58	5	False
46 41	Zubat	Poison	Flying	40	45	35	30	40	55	1	False
631 570	Zorua	Dark	NaN	40	65	40	80	40	65	5	False
632 571	Zoroark	Dark	NaN	60	105	60	120	60	105	5	False
...	...	...	...	...	...	...	...	...	...	...	...
393 359	AbsolMega Absol	Dark	NaN	65	150	60	115	60	115	3	False
392 359	Absol	Dark	NaN	65	130	60	75	60	75	3	False
68 63	Abra	Psychic	NaN	25	20	15	105	55	90	1	False
511 460	AbomasnowMega Abomasnow	Grass	Ice	90	132	105	132	105	30	4	False
510 460	Abomasnow	Grass	Ice	90	92	75	92	85	60	4	False

800 rows x 12 columns

Sorting: Multiple Columns

Column 'Type 1': Ascending

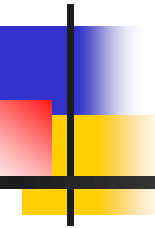
Column 'HP': Descending

```
df.sort_values(['Type 1', 'HP'], ascending=[1,0])
```

df

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True
796	719	DiancieMega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True
797	720	HoopaHoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True
798	720	HoopaHoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True
799	721	Volcanion	Fire	Water	80	110	120	130	90	70	6	True

800 rows × 12 columns



Making Changes to the Data

Add Another Column with the Sum of Numbers

```
df['Total'] = df['HP'] + df['Attack'] + df['Defense'] + df['Sp. Atk'] + df['Sp. Def'] + df['Speed']  
df
```

#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total	
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False	309
...	...	...	...	...	...	...	...	...	...	...	...	...	
795	719	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True	600
796	719	DiancieMega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True	700
797	720	HoopaHoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True	600
798	720	HoopaHoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True	680
799	721	Volcanion	Fire	Water	80	110	120	130	90	70	6	True	600

800 rows x 13 columns

```
45+49+49+65+65+45
```

318

Delete a Column

```
df = df.drop(columns=['Total'])
```

```
df
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True
796	719	DiancieMega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True
797	720	HoopaHoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True
798	720	HoopaHoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True
799	721	Volcanion	Fire	Water	80	110	120	130	90	70	6	True

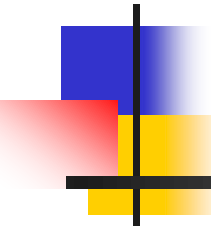
800 rows x 12 columns

Second Method Using Axis

```
df['Total'] = df.iloc[:, 4:10].sum(axis=1)
df
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False	309
...	...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True	600
796	719	DiancieMega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True	700
797	720	HoopaHoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True	600
798	720	HoopaHoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True	680
799	721	Volcanion	Fire	Water	80	110	120	130	90	70	6	True	600

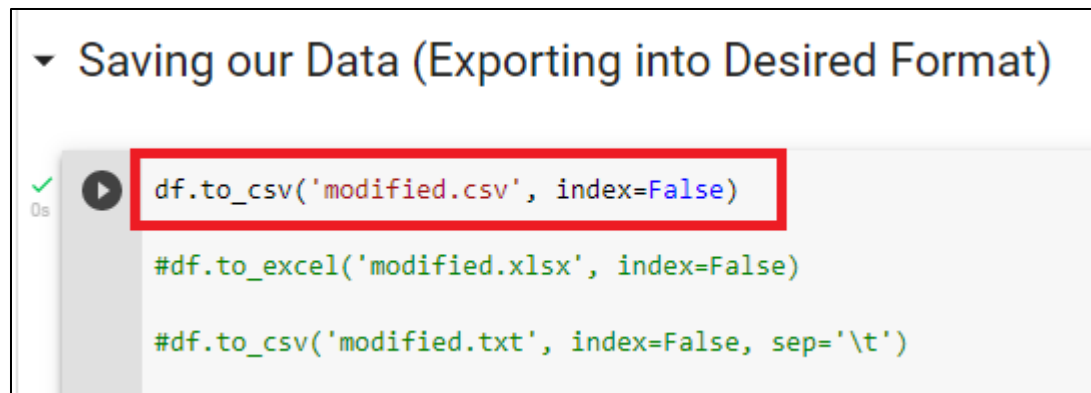
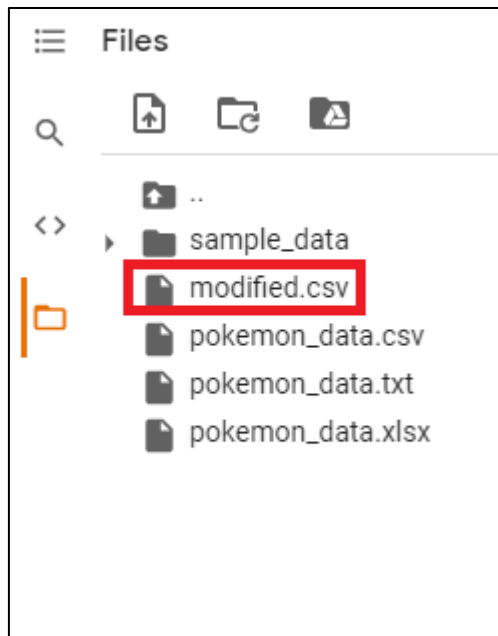
800 rows x 13 columns

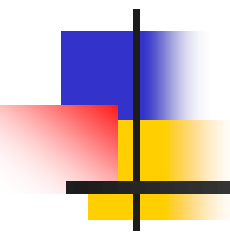


Saving Data Frame to a File

Saving a Data Frame

.csv format (comma separated or tab separated)
Excel (.xlsx)





Filter Data

Select Boolean Condition

Select Where 'Type 1' = 'Grass'

```
df.loc[(df['Type 1'] == 'Grass')]
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
48	43	Oddish	Grass	Poison	45	50	55	75	65	30	1	False	320
...	...	...	...	...	...	...	...	...	...	...	...	...	...
718	650	Chespin	Grass	NaN	56	61	65	48	45	38	6	False	313
719	651	Quilladin	Grass	NaN	61	78	95	56	58	57	6	False	405
720	652	Chesnaught	Grass	Fighting	88	107	122	74	75	64	6	False	530
740	672	Skiddo	Grass	NaN	66	65	48	62	57	52	6	False	350
741	673	Gogoat	Grass	NaN	123	100	62	97	81	68	6	False	531

70 rows x 13 columns

Multiple Boolean Conditions

OLD Index Values

```
df.loc[(df['Type 1'] == 'Grass') & (df['Type 2'] == 'Poison') & (df['HP'] > 70)]
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	490
77	71	Victreebel	Grass	Poison	80	105	65	100	70	70	1	False	490
652	591	Amoonguss	Grass	Poison	114	85	70	85	80	30	5	False	464

Multiple Boolean Conditions

New Index Values

```
[21] new_df = df.loc[(df['Type 1'] == 'Grass') & (df['Type 2'] == 'Poison') & (df['HP'] > 70)]
new_df
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	490
77	71	Victreebel	Grass	Poison	80	105	65	100	70	70	1	False	490
652	591	Amoonguss	Grass	Poison	114	85	70	85	80	30	5	False	464

```
new_df.reset_index(drop=True, inplace=True)
new_df
#new_df.to_csv('filtered.csv')
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525
1	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625
2	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	490
3	71	Victreebel	Grass	Poison	80	105	65	100	70	70	1	False	490
4	591	Amoonguss	Grass	Poison	114	85	70	85	80	30	5	False	464

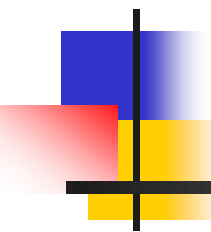
If Total > 500: Change

Generation + Legendary = Test 1 + Test 2

```
df.loc[df['Total'] > 500, ['Generation', 'Legendary']] = ['Test 1', 'Test 2']  
df
```

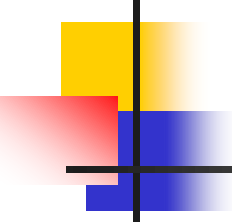
	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	Test 1	Test 2	525
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	Test 1	Test 2	625
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False	309
...	...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	50	100	150	100	150	50	Test 1	Test 2	600
796	719	DiancieMega Diancie	Rock	Fairy	50	160	110	160	110	110	Test 1	Test 2	700
797	720	HoopaHoopa Confined	Psychic	Ghost	80	110	60	150	130	70	Test 1	Test 2	600
798	720	HoopaHoopa Unbound	Psychic	Dark	80	160	60	170	130	80	Test 1	Test 2	680
799	721	Volcanion	Fire	Water	80	110	120	130	90	70	Test 1	Test 2	600

800 rows x 13 columns



Groupby

Identical to Pivot Tables Excel



Simple Example of Groupby

- Groupby Department (Categorical)
 - Count Budget (Numerical)
 - A = 4
 - B = 2
 - Sum Budget (Numerical)
 - A = 1,200
 - B = 500
 - Average Budget (Numerical)
 - A = 300
 - B = 250

	A	B	C	D
1	Name	Dept	Budget	
2	John	A	100	
3	Steve	B	200	
4	Kathy	B	300	
5	Cindi	A	500	
6	Larry	A	50	
7	Gavin	A	550	
8				
9				

Simple Example of Groupby

- Groupby Department (Categorical)
 - Count Budget (Numerical)
 - A = 4
 - B = 2
 - Sum Budget (Numerical)
 - A = 1,200
 - B = 500
 - Average Budget (Numerical)
 - A = 300
 - B = 250

```
[1] import pandas as pd

[2] from google.colab import files
files.upload()

Choose Files Pandas_Groupby_example_Data.csv
• Pandas_Groupby_example_Data.csv(application/vnd.ms-excel)
Saving Pandas_Groupby_example_Data.csv to Pandas_Groupby
{'Pandas_Groupby_example_Data.csv': b'Name,Dept,Budget\r\n'}

df = pd.read_csv('Pandas_Groupby_example_Data.csv')
print(df.head(6))
```

	Name	Dept	Budget
0	John	A	100
1	Steve	B	200
2	Kathy	B	300
3	Cindi	A	500
4	Larry	A	50
5	Gavin	A	550

```
[4] df.groupby('Dept').count()['Budget']
```

```
Dept
A    4
B    2
Name: Budget, dtype: int64
```

```
df.groupby('Dept').sum()['Budget']
```

```
Dept
A   1200
B    500
Name: Budget, dtype: int64
```

```
df.groupby('Dept').mean()['Budget']
```

```
Dept
A    300
B    250
Name: Budget, dtype: int64
```

Group By: Pokeman Dataset

```
[29] df = pd.read_csv('modified.csv')
```

```
df.groupby(['Type 1']).mean().sort_values('Defense',ascending=False)
```

	#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Type 1										
Steel	442.851852	65.222222	92.703704	126.370370	67.518519	80.629630	55.259259	3.851852	0.148148	487.703704
Rock	392.727273	65.363636	92.863636	100.795455	63.340909	75.477273	55.909091	3.454545	0.090909	453.750000
Dragon	474.375000	83.312500	112.125000	86.375000	96.843750	88.843750	83.031250	3.875000	0.375000	550.531250
Ground	356.281250	73.781250	95.750000	84.843750	56.468750	62.750000	63.906250	3.156250	0.125000	437.500000
Ghost	486.500000	64.437500	73.781250	81.187500	79.343750	76.468750	64.343750	4.187500	0.062500	439.562500
Water	303.089286	72.062500	74.151786	72.946429	74.812500	70.517857	65.964286	2.857143	0.035714	430.455357
Ice	423.541667	72.000000	72.750000	71.416667	77.541667	76.291667	63.458333	3.541667	0.083333	433.458333
Grass	344.871429	67.271429	73.214286	70.800000	77.500000	70.428571	61.928571	3.357143	0.042857	421.142857
Bug	334.492754	56.884058	70.971014	70.724638	53.869565	64.797101	61.681159	3.217391	0.000000	378.927536
Dark	461.354839	66.806452	88.387097	70.225806	74.645161	69.516129	76.161290	4.032258	0.064516	445.741935
Poison	251.785714	67.250000	74.678571	68.821429	60.428571	64.392857	63.571429	2.535714	0.000000	399.142857
Fire	327.403846	69.903846	84.769231	67.769231	88.980769	72.211538	74.442308	3.211538	0.096154	458.076923
Psychic	380.807018	70.631579	71.456140	67.684211	98.403509	86.280702	81.491228	3.385965	0.245614	475.947368
Electric	363.500000	59.795455	69.090909	66.295455	90.022727	73.704545	84.500000	3.272727	0.090909	443.409091
Flying	677.750000	70.750000	78.750000	66.250000	94.250000	72.500000	102.500000	5.500000	0.500000	485.000000
Fighting	363.851852	69.851852	96.777778	65.925926	53.111111	64.703704	66.074074	3.370370	0.000000	416.444444
Fairy	449.529412	74.117647	61.529412	65.705882	78.529412	84.705882	48.588235	4.117647	0.058824	413.176471

✓ Os completed at 3:32 PM

Pokeman Dataset

Group By: Type 1 + Type 2

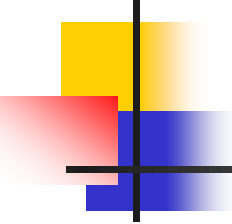
```
[34] df['count'] = 1
```

```
df.head()
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total	count
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	318	1
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	405	1
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	525	1
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	625	1
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False	309	1

```
df.groupby(['Type 1', 'Type 2']).count()['count']
```

```
└─ Type 1  Type 2  count
  Bug      Electric  2
           Fighting  2
           Fire      2
           Flying    14
           Ghost     1
           ..
  Water    Ice       3
           Poison    3
           Psychic   5
           Rock      4
           Steel     1
Name: count, Length: 136, dtype: int64
```



Summary

- What is Pandas?
- Pandas Functions
- Why Pandas?
- Dataset + Load Data in Pandas' Data Frame
- Read data from a Data Frame
- Sorting Data
- Make Changes to Data
- Saving Data Frame to a File
- Filter Data Select Boolean Conditions
- Groupby Clause (Pivot Table)

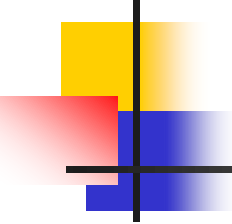
CS501

Introductory Computation for Scientists



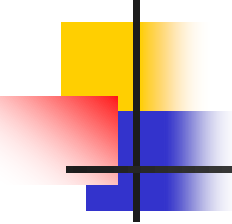
Dr. Ash Pahwa

Lesson 7.1: Pandas DataFrame from Dictionary



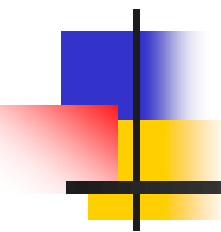
Outline

- Raw Data
- Creating a DataFrame
- Accessing data from a DataFrame
- Statistical Information about Data
- Sorting Data



Raw Data: Student Test Scores

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85



Creating a DataFrame

Raw Data in a Dictionary

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

▼ Loading data into Pandas

```
[1] import pandas as pd
```

▼ Data is given column wise in a dictionary

```
[2] grades_dict = {'Wally': [87, 96, 70], 'Eva': [100, 87, 90], 'Sam': [94, 77, 90], 'Katie': [100, 81, 82], 'Bob': [83, 65, 85]}
```

Load Dictionary Data in a Pandas DataFrame

1. Creating a DataFrame from a Dictionary

```
✓ [4] grades = pd.DataFrame(grades_dict)
0s print(grades)
```

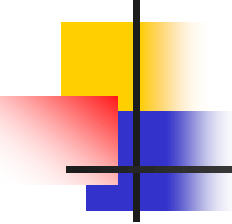
	Wally	Eva	Sam	Katie	Bob
0	87	100	94	100	83
1	96	87	77	81	65
2	70	90	90	82	85

```
✓ grades.index = index = ['Test1', 'Test2', 'Test3']
0s grades
```

```
➤
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Transpose of a Data



	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

```
[ ] grades.T
```

	Test1	Test2	Test3
Wally	87	96	70
Eva	100	87	90
Sam	94	77	90
Katie	100	81	82
Bob	83	65	85

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

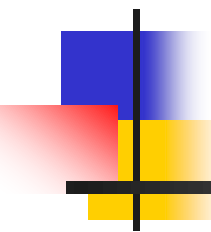
Customizing a DataFrame's indices with the index Attribute

▼ 2. Customizing a DataFrame's indices with the index Attribute

```
[ ] grades = pd.DataFrame(grades_dict, index = ['Test1', 'Test2', 'Test3'])  
print(grades)
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85



Accessing Data from a DataFrame

Accessing a DataFrame Column

▼ 3. Accessing a DataFrame's Columns

```
[ ] grades['Eva']
```

```
Test1    100  
Test2     87  
Test3     90  
Name: Eva, dtype: int64
```

```
[ ] grades.Sam
```

```
Test1     94  
Test2     77  
Test3     90  
Name: Sam, dtype: int64
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Selecting Rows of a DataFrame using 'loc' and 'iloc' methods

▼ 4. Selecting Rows via loc and iloc Attributes

```
[ ] grades.loc['Test1']
```

```
Wally      87
Eva       100
Sam        94
Katie     100
Bob        83
Name: Test1, dtype: int64
```

```
▶ grades.iloc[1]
```

```
Wally      96
Eva        87
Sam        77
Katie      81
Bob        65
Name: Test2, dtype: int64
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Selecting Rows via Slices and Lists with the loc and iloc Attributes

▼ 5. Selecting Rows via Slices and Lists with the loc and iloc Attributes

```
[ ] grades.loc['Test1':'Test3']
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

```
[ ] grades.iloc[0:2]
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Selecting Rows via Slices and Lists with the loc and iloc Attributes

Feed List of a List

```
grades.loc['Test1','Test3']
```

```
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py in get_loc
    3360         try:
-> 3361             return self._engine.get_loc(casted_key)
    3362         except KeyError as err:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

```
[ ] grades.loc[['Test1','Test3']]
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test3	70	90	90	82	85

```
[ ] grades.iloc[0,2]
```

94

```
grades.iloc[[0,2]]
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test3	70	90	90	82	85

Select Subset of Rows and Columns

▼ 6. Selecting Subsets of the Rows and Columns

```
▶ grades.loc['Test1':'Test2',['Eva', 'Katie']]
```

👤

	Eva	Katie
Test1	100	100
Test2	87	81

```
[ ] grades.iloc[[0,2],0:3]
```

	Wally	Eva	Sam
Test1	87	100	94
Test3	70	90	90

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Boolean Masking

7. Boolean Indexing

```
[ ] grades[grades >= 90]
```

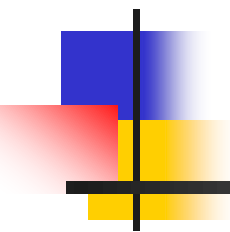
	Wally	Eva	Sam	Katie	Bob
Test1	NaN	100.0	94.0	100.0	NaN
Test2	96.0	NaN	NaN	NaN	NaN
Test3	NaN	90.0	90.0	NaN	NaN

```
[ ] grades[(grades >= 80) & (grades < 90)]
```

	Wally	Eva	Sam	Katie	Bob
Test1	87.0	NaN	NaN	NaN	83.0
Test2	NaN	87.0	NaN	81.0	NaN
Test3	NaN	NaN	NaN	82.0	85.0

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Statistical Information About Data



Statistics

8. Descriptive Statistics

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

grades.describe()

	Wally	Eva	Sam	Katie	Bob
count	3.000000	3.000000	3.000000	3.000000	3.000000
mean	84.333333	92.333333	87.000000	87.666667	77.666667
std	13.203535	6.806859	8.888194	10.692677	11.015141
min	70.000000	87.000000	77.000000	81.000000	65.000000
25%	78.500000	88.500000	83.500000	81.500000	74.000000
50%	87.000000	90.000000	90.000000	82.000000	83.000000
75%	91.500000	95.000000	92.000000	91.000000	84.000000
max	96.000000	100.000000	94.000000	100.000000	85.000000

Set Precision = 2

```
[ ] pd.set_option('precision',2)
```

```
▶ grades.describe()
```

	Wally	Eva	Sam	Katie	Bob
count	3.00	3.00	3.00	3.00	3.00
mean	84.33	92.33	87.00	87.67	77.67
std	13.20	6.81	8.89	10.69	11.02
min	70.00	87.00	77.00	81.00	65.00
25%	78.50	88.50	83.50	81.50	74.00
50%	87.00	90.00	90.00	82.00	83.00
75%	91.50	95.00	92.00	91.00	84.00
max	96.00	100.00	94.00	100.00	85.00

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Statistics of the Transpose Matrix

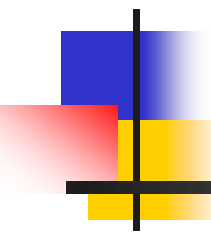
	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

grades.T

	Test1	Test2	Test3
Wally	87	96	70
Eva	100	87	90
Sam	94	77	90
Katie	100	81	82
Bob	83	65	85

grades.T.describe()

	Test1	Test2	Test3
count	5.00	5.00	5.00
mean	92.80	81.20	83.40
std	7.66	11.54	8.23
min	83.00	65.00	70.00
25%	87.00	77.00	82.00
50%	94.00	81.00	85.00
75%	100.00	87.00	90.00
max	100.00	96.00	90.00



Sorting Data

Sorting by Rows by Their Indices

▼ 10. Sorting by Rows by Their Indices

✓ 0s  `grades.sort_index(ascending=False)`

	Wally	Eva	Sam	Katie	Bob
Test3	70	90	90	82	85
Test2	96	87	77	81	65
Test1	87	100	94	100	83

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

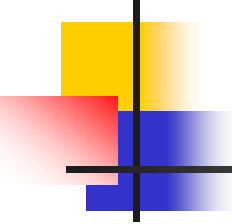
Sorting by Column Indices

▼ 11. Sorting by Column Indices

```
grades.sort_index(axis=1)
```

	Bob	Eva	Katie	Sam	Wally
Test1	83	100	100	94	87
Test2	65	87	81	77	96
Test3	85	90	82	90	70

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85



Sort by Column Values

```
[ ] grades.sort_values(by='Test1', axis=1, ascending=False)
```

	Eva	Katie	Sam	Wally	Bob
Test1	100	100	94	87	83
Test2	87	81	77	96	65
Test3	90	82	90	70	85

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

Sort by Column Values

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

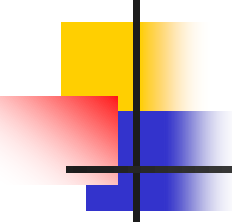
Transpose and then Sort

```
[ ] grades.T.sort_values(by='Test1', ascending=False)
```

	Test1	Test2	Test3
Eva	100	87	90
Katie	100	81	82
Sam	94	77	90
Wally	87	96	70
Bob	83	65	85

```
[ ] grades.loc['Test1'].sort_values(ascending=False)
```

```
Eva      100
Katie     100
Sam       94
Wally     87
Bob       83
Name: Test1, dtype: int64
```



Summary

- Raw Data
- Creating a DataFrame
- Accessing data from a DataFrame
- Statistical Information about Data
- Sorting Data